有序存储方案

ghostwydwu（武勇栋）

大纲

- 适用场景
- 接口
- 架构
- 元数据
- 副本同步
- 存储引擎
- 资源调度
- 故障探测&恢复

适用场景

✓**树形结构——索引目录**
Key：QQ + 上级目录ID + 目录/文件ID

✓**一纬时间序——Feed**
Key：QQ + 时间

●单条插入，按范围批量拉取

●业务无需对记录打包，无放大流量问题

✓**二维空间序——LBS类应用**
Key：GeoHash（long，lat）

●减少批量拉取的IO，优化性能

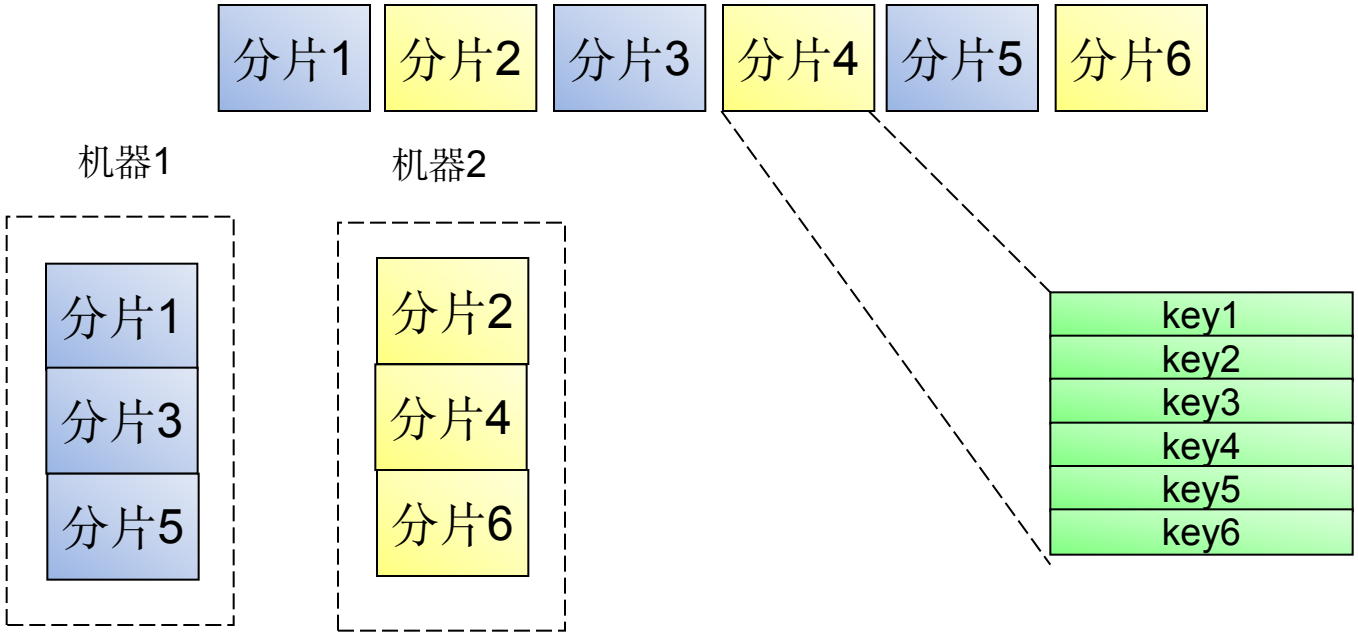
接口

- 单记录接口
  - Get（Key， Value）
  - Set（Key， Value）
  - Del（Key）

- 批量有序key接口
  - GetBatch（Begin_key, End_key, Value_list)
  - SetBatch（Key_value_list）
  - DelBatch（ Begin_key, End_key）

- Key value最大长度限制
  - Key：推荐使用1KB以下的key长度，最大支持使用10KB
  - Value：最大长度支持512KB
  - 批量接口最大拉取数据量：2MB

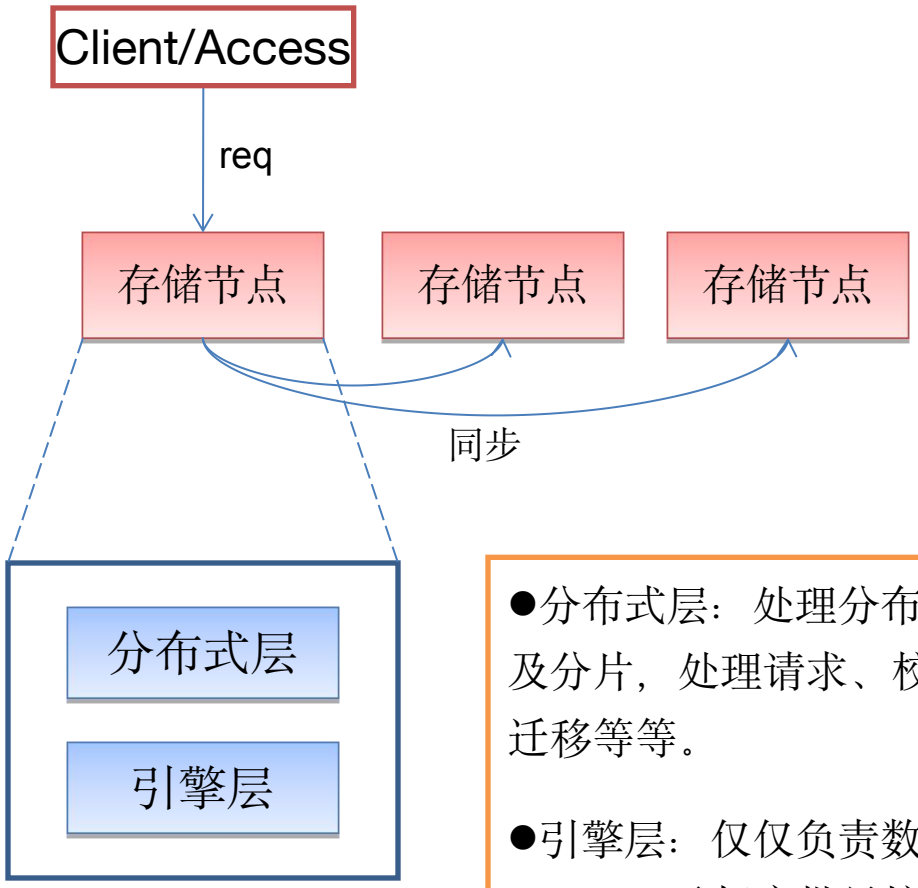
基本思路



两个基本问题：

1. 如何根据请求key的range找到对应的分片，机器和磁盘——路由
2. 如何将分片内的key按有序的方式组织起来存储——存储引擎

架构-模块结构



资源管理

资源调度

●分布式层：处理分布式相关事务，理解大表及分片，处理请求、校验路由、同步、统计、迁移等等。

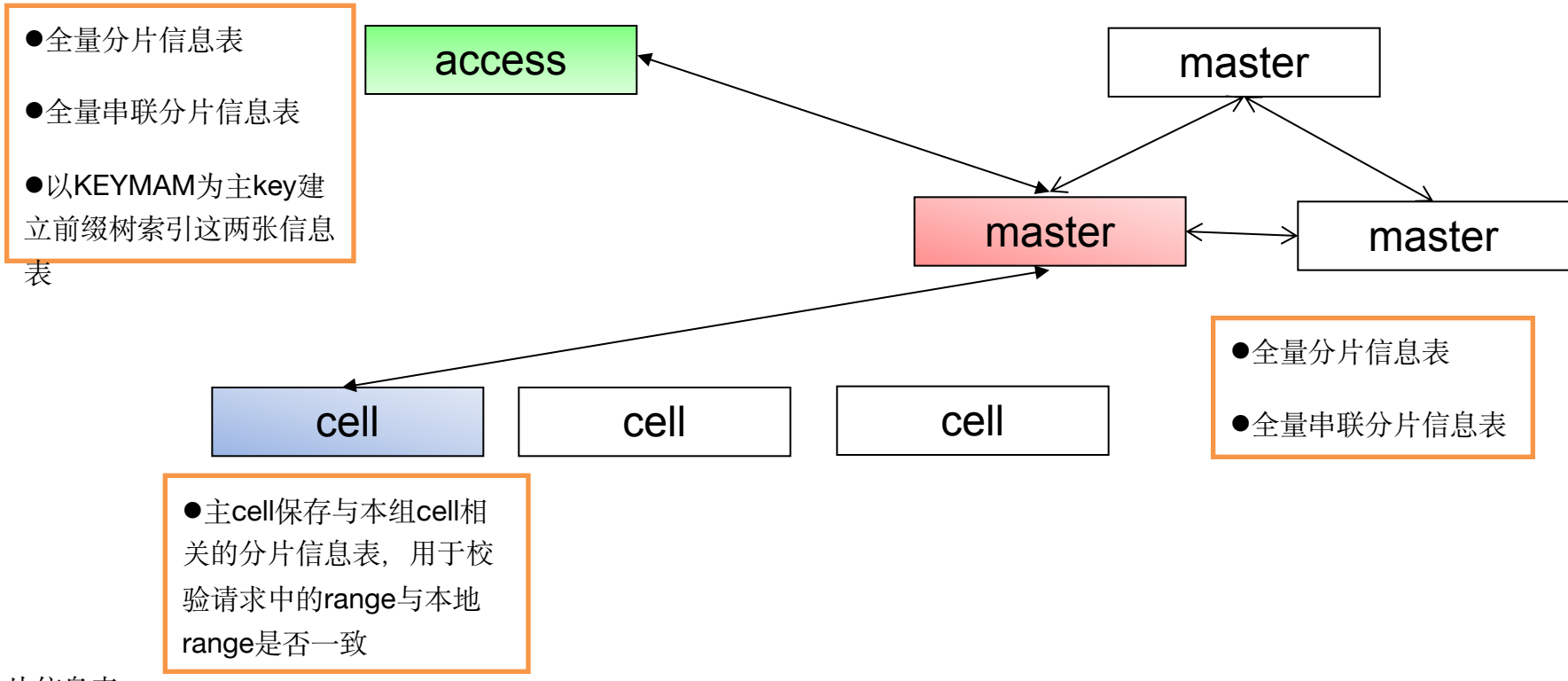
●引擎层：仅仅负责数据的存储。提供set, get，del及相应批量接口。

元数据

- 路由元数据
  - 结构：按key范围进行逻辑分片（sharding）
  - 存储：以将分片信息和迁移时产生的串联路由分片以放入共享内存，同时以tree的方式建立索引便于查找
  - 容灾：3台master机器运行raft协议进行容灾

路由架构（1）——分片

- 结构：按key范围进行逻辑分片（sharding）
- 容灾：raft协议，3台master进行容灾



分片信息表

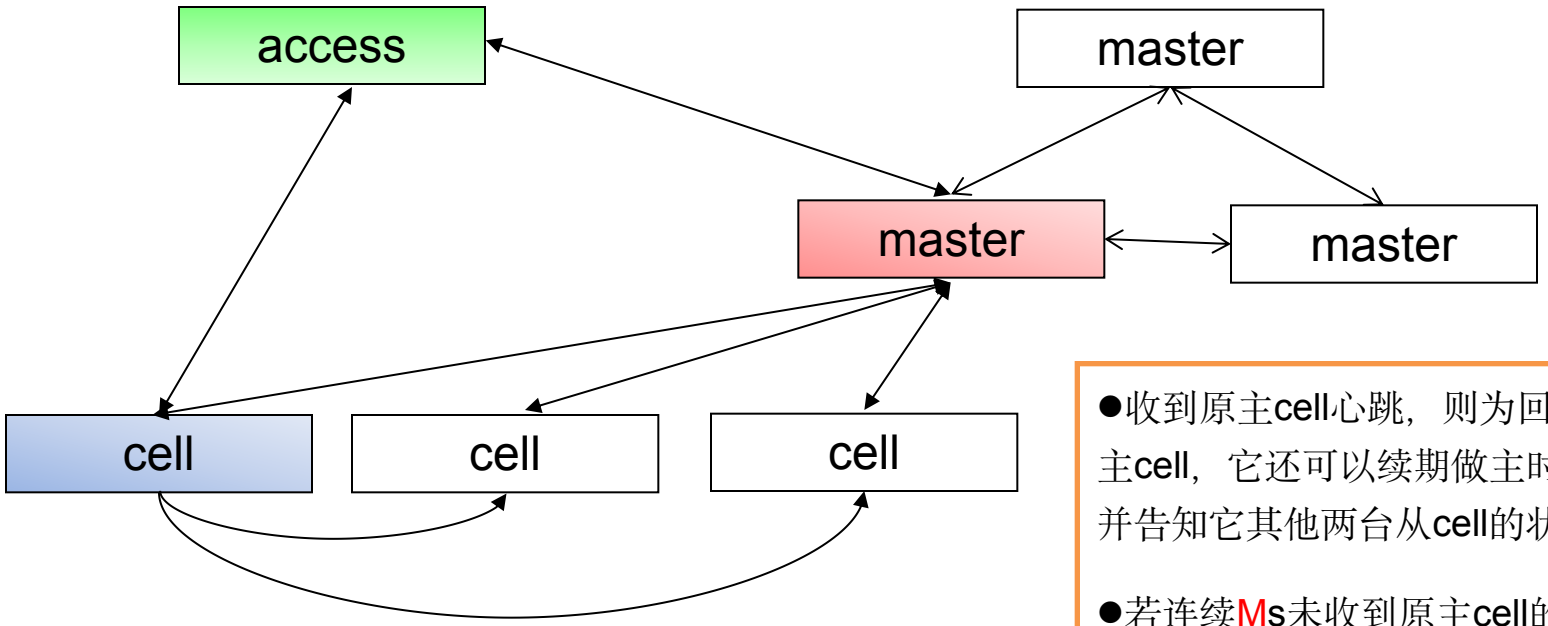
分片id	seq	keylen	keymax	主ip	磁盘	从Ip1	从ip1 state	从ip2	从ip2 state	有效位	保留

串联分片信息表

分片id	seq	keylen	keymax	源ip1	源磁盘 1	源ip2	源磁盘 2	目标ip1	目标磁盘1	目标ip2	目标磁盘2

路由架构（2）——主从切换

●定期与master同步主cell的ip



●3台cell每Ts向master发送抢主心跳

●主cell维护一个item，向其他cell同步请求时，带上item号，发现其他cell的item号比自己的还大，则返回请求失败，并取消自己的主资格

●只有主cell可以处理access的请求，从cell拒绝access的所有请求

●收到原主cell心跳，则为回包告诉主cell，它还可以续期做主时间Ms，并告知它其他两台从cell的状态

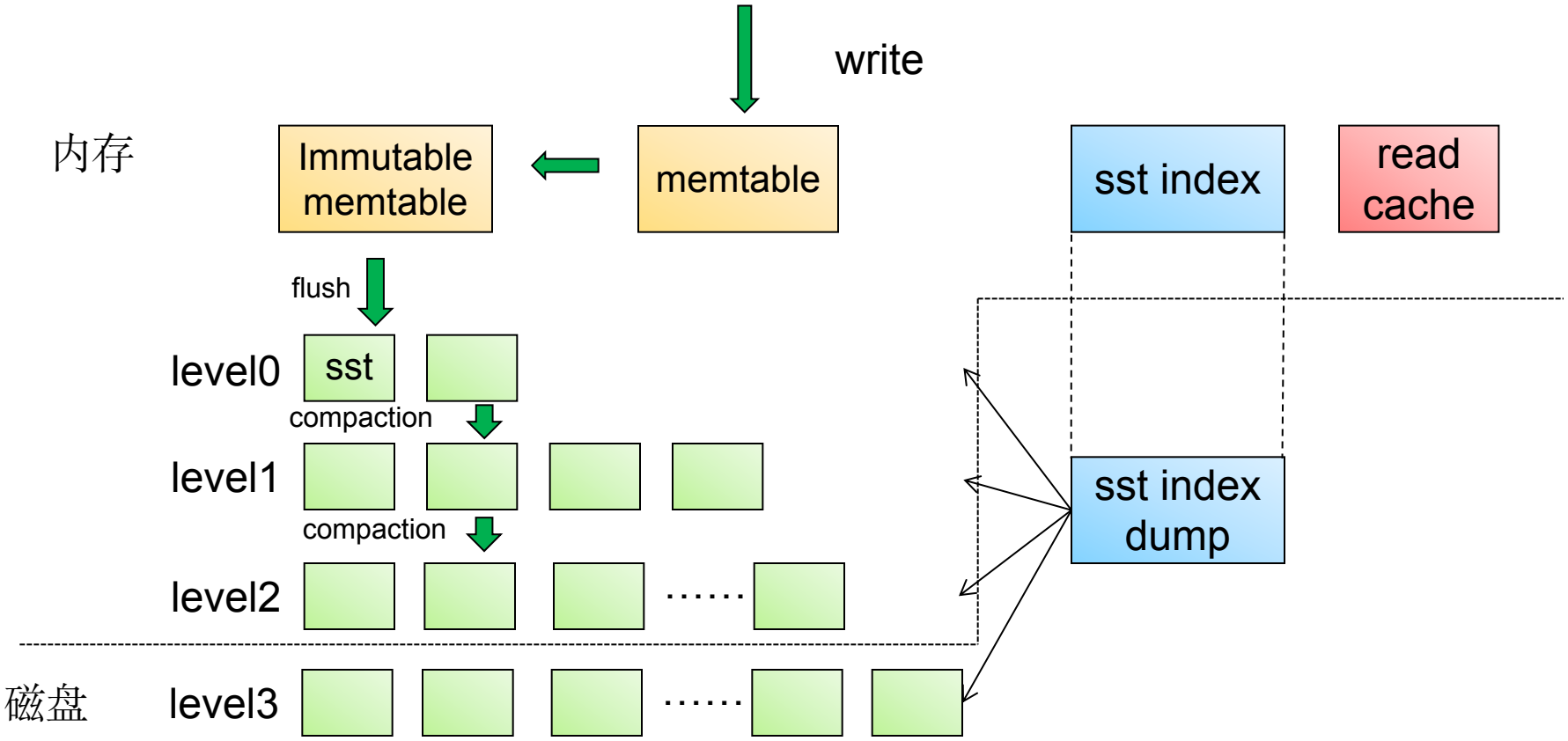
●若连续Ms未收到原主cell的心跳包，则下次从cell抢主心跳包达到时，切换从cell为新主，并将item+1。

●若连续Ms未收到从cell的抢主心跳包，则将从cell状态置为dead

副本同步和主从切换

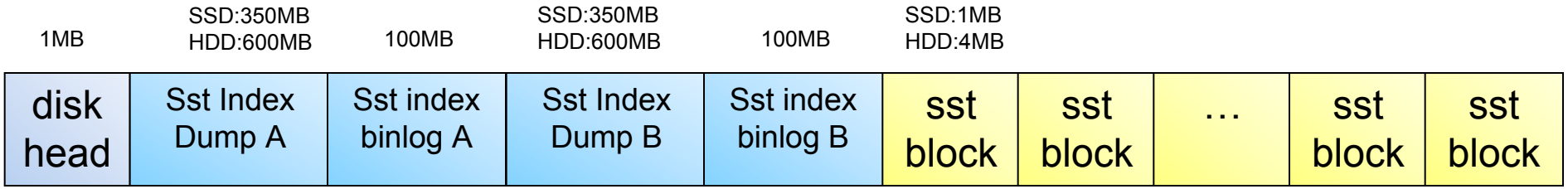
- 分布
  - 固定组对/全局
- 同步（细化）
  - 同步点：存储节点
  - 同步一致性协议：all，版本号

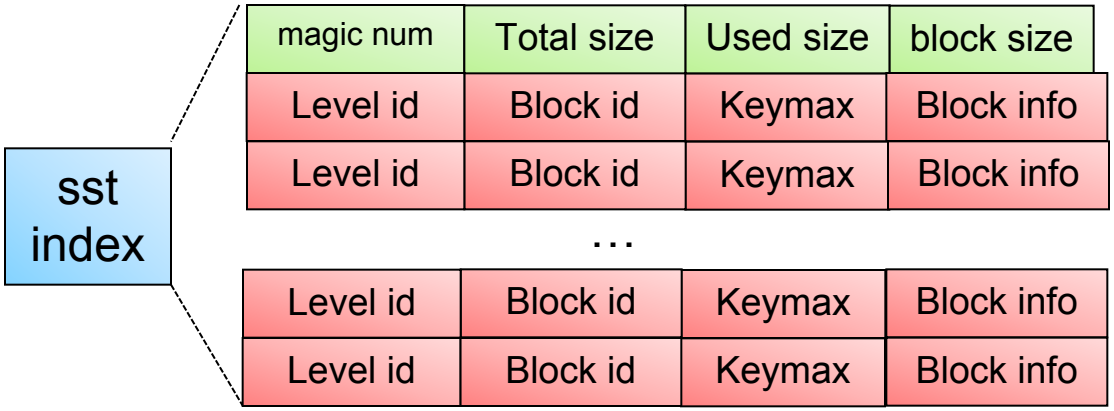
存储引擎——结构



sst block：顺序存放key-value
sst index：存放了每一个sst block所包含的key范围，以及其所属的level
Memtable：write buffer，buffer中的key-value使用skip-list有序存放
Immutable memtable：memtable 被写满后，变成immutable table，并被淘汰到磁盘
Read cache：读缓存

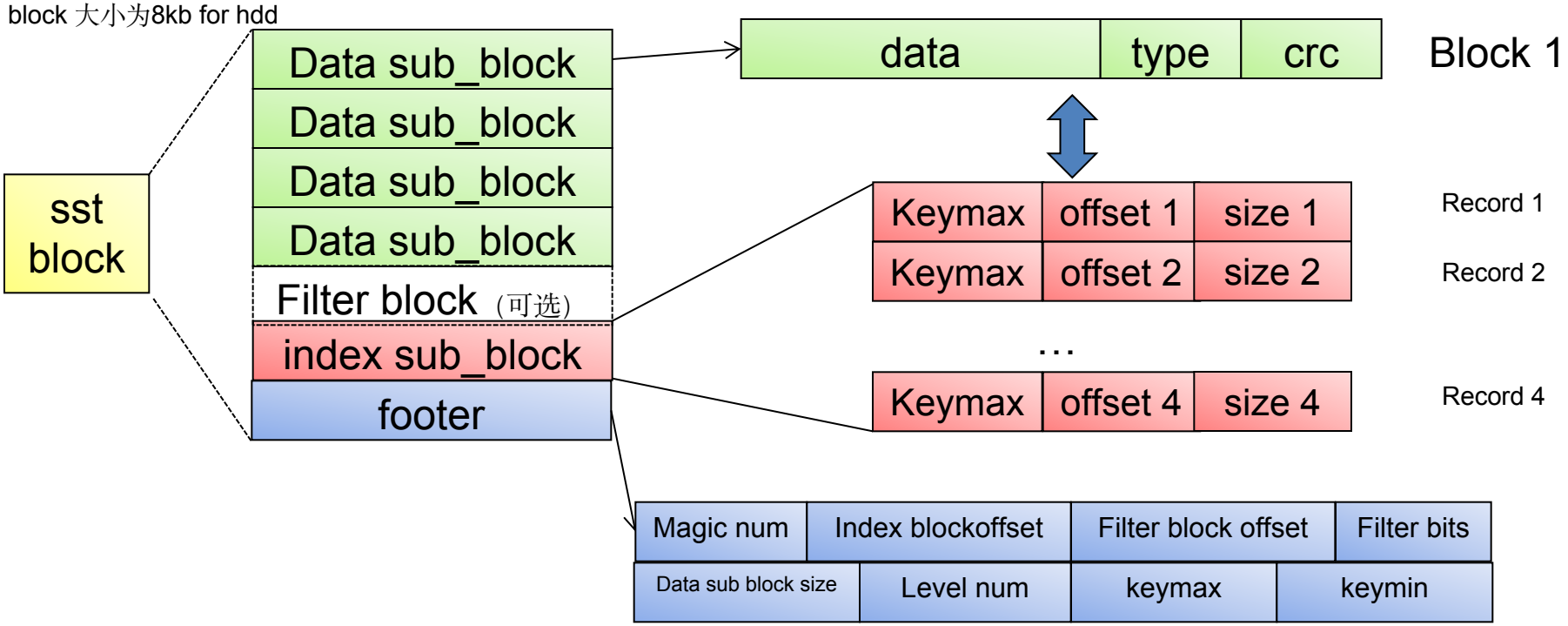
存储引擎——磁盘结构（1）

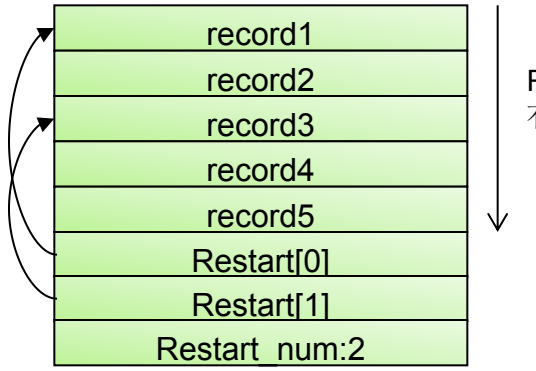

1MB
SSD:350MB
HDD:600MB
100MB
SSD:350MB
HDD:600MB
100MB
SSD:1MB
HDD:4MB
disk head
Sst Index Dump A
Sst index binlog A
Sst Index Dump B
Sst index binlog B
sst block
sst block
…
sst block
sst block


sst index
magic num
Total size
Used size
block size
Level id
Block id
Keymax
Block info
Level id
Block id
Keymax
Block info
…
Level id
Block id
Keymax
Block info
Level id
Block id
Keymax
Block info

存储引擎——磁盘结构（2）

Sub block 大小为8kb for ssd
Sub block 大小为8kb for hdd





Record
有序

data sub block和index sub block格式



Record 格式

读上来数据找对应的record位置的时候，利用restart做二分查找

存储引擎——磁盘结构（3）

Bloom filter（可选）
每个sst block实现一个bloom filter

m	bit数组的宽度（bit数）
n	加入其中的key的数量
k	使用的hash函数的个数
f	False Positive的比率

其中：

$$m \geq n \frac{\log_2(1/\epsilon)}{\ln 2} = n \log_2 e \cdot \log_2(1/\epsilon).$$

k = ln2· (m/n)

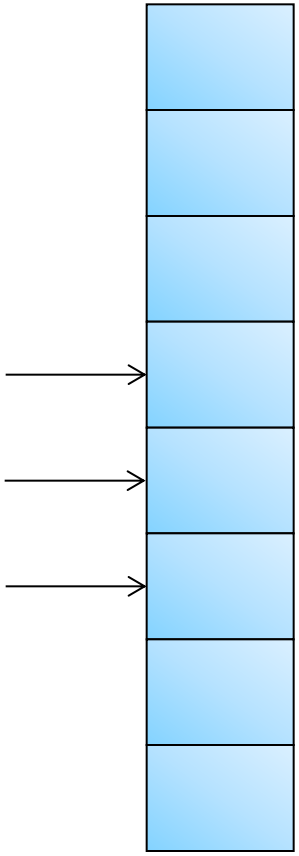
若取平均记录大小为300字节（相册业务），则每4MB中key的个数，为13600个

e	m	k
0.5	2500字节	1
0.31	4096KB	1
0.1	8192KB	3
0.04	12KB	3

SSD盘：选用2k字节/MB作为bloom filter，对应的误报率为0.1
HDD盘：选用12K字节/4MB节作为bloom filter，对应的误报率为0.04

存储引擎——磁盘结构（4）

当需要查找的index sub_block 大小范围超过64KB时，采用二分查找采用单次读取4kb，分多次读取的方式寻找对应的data sub block



Key size	Sub index block size	读次数
16字节	8kb	1次8kb+1次8kb
32字节	8kb	1次8kb+1次8kb
64字节	8kb	1次8kb+1次8kb
128字节	16kb	1次16kb+1次8kb
1K字节	128Kb	2次4kb+1次64kb+1次8kb

存储引擎——磁盘结构（5）

- 对于**SSD**磁盘
- 大于**64kb**的**block**索引块，使用二分查找减少读取的数据量有明显降低读取延时的效果。
- 在大部分场景下（**key<64**字节，**value<4kb**）,存储引擎能够达到**1w**次**iops**的读取性能

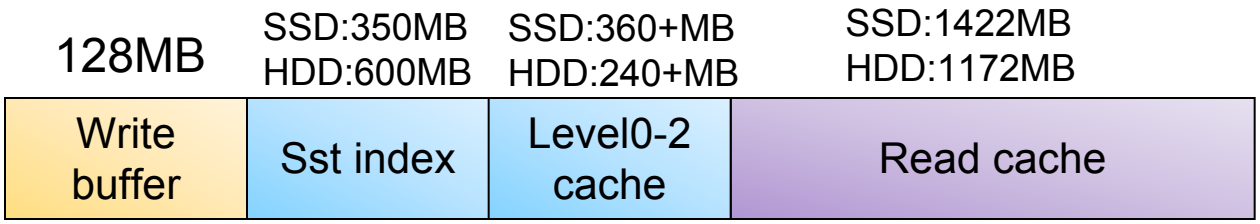
读取文件测试 1MB SST 8KB sub block										
key长度	索引块大小		耗时（us）		耗时（us）		耗时（us）		耗时（us）	iops
1024	128KB			1次128kb读	1773	1次8kb读	1307	总共	3080	2597.402597
1024	128KB	2次4kb读	790	1次64kb读	792	1次8kb读	510	总共	2092	3824.091778
128	16kb			1次64kb读	1086	1次8kb读	729	总共	1815	4407.713499
64	8kb			1次8kb读	295	1次8kb读	295	总共	590	13559.32203
32	8kb			1次8kb读	295	1次8kb读	295	总共	590	13559.32203
16	8kb			1次8kb读	295	1次8kb读	295	总共	590	13559.32203

- 对于**SATA**磁盘
- 采用**4MB**的**sst block** 和 **256kb** 大小的**data sub_block**

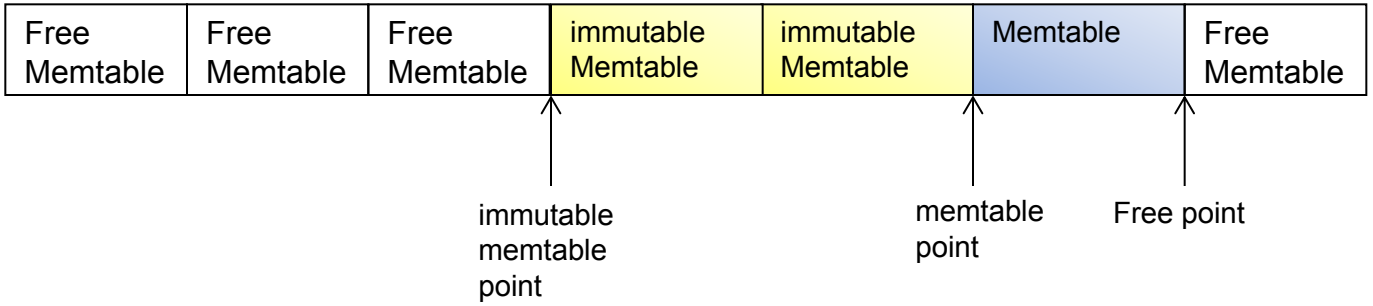
读取文件测试 4MB SST 256KB sub block								
key长度	索引块大小		耗时（us）		耗时（us）		耗时（us）	iops
1024	16KB	1次16kb读	5337	1次256kb读	3626	总共	8963	111.5698
128	4kb	1次4kb读	5274	1次256kb读	3721	总共	8995	111.1729
64	4kb	1次4kb读	5274	1次256kb读	3721	总共	8995	111.1729
32	4kb	1次4kb读	5274	1次256kb读	3721	总共	8995	111.1729
16	4kb	1次4kb读	5274	1次256kb读	3721	总共	8995	111.1729

存储引擎——内存结构（1）

- 总体分布

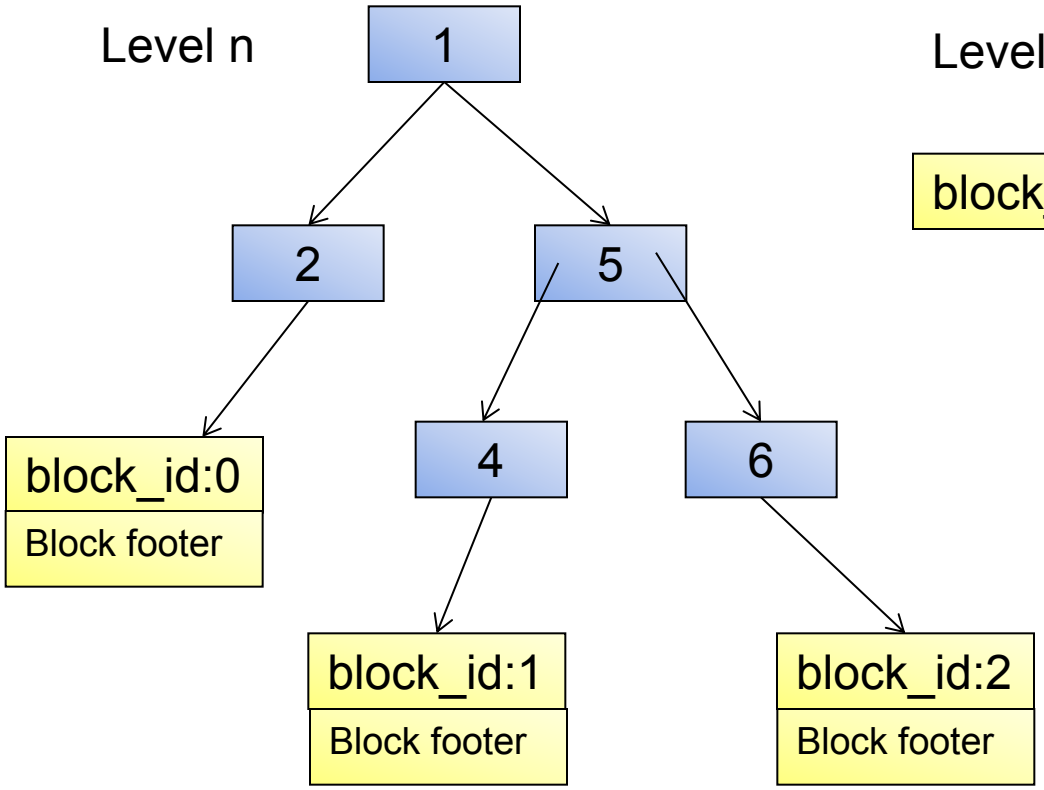


- Write buffer
  - •大小：128MB，每个memtable 4MB
  - •Memtable：使用skip-list有序存放kv
  - •Immutable memtable：memtable 被写满后，变成immutable memtable，并被淘汰到磁盘
  - •使用一个循环链表来管理memtable和immutable



存储引擎——内存结构（2）

- sst index
  - 大小为350Mb
  - 磁盘中sst index的镜像。
  - 以keymax为key每一个level（level0除外），针对自己的sst block建立一个10叉的b+树做索引。

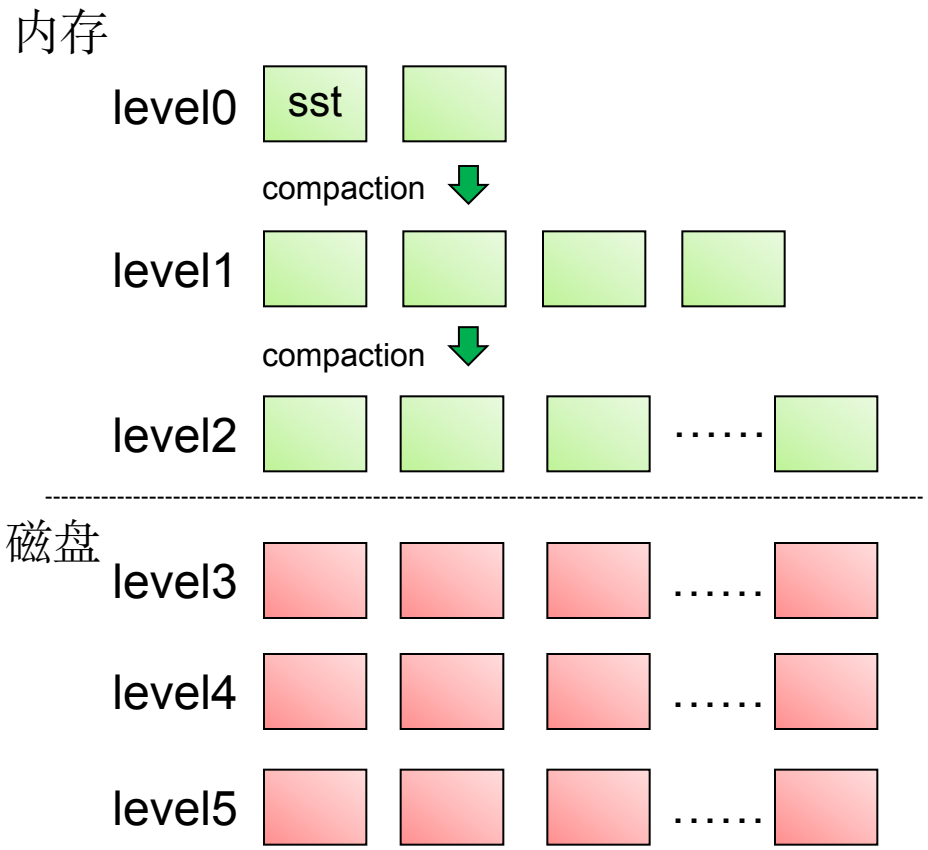


Level 0

block_id	keymin	keymax

- block footer是指sst block的footer中除了keymin和keymax以外的其他信息

存储引擎——内存结构（3）



- level0-level2的sst block的实时数据放在内存中。
- 优势：
  - 在写io较大时，相对于原始的leveldb可以降低10倍的写放大
  - 随机写性能提升50%
- 劣势：
  - 磁盘掉电时最多丢掉level0-2的最长轮转时间+memtable最长合并时间内写入的数据(约10 min)的数据。（内存中最大600MB数据，若合并带宽为1MB/s，则大约10min左右的数据会丢失）

存储引擎——内存结构（3）

- Read Cache
  - Read Cache的粒度为SST block。
  - 以sst block id和sub block id为key做cache缓存磁盘中的内容
  - Hash 实现索引（cache access 实现）
  - Cache对应的block改变时，drop cache
  - 目前暂时采用LRU做淘汰

存储引擎——内存结构（4）

- Read Cache分配策略

SSD:
HDD:

	Bloom filter	Index sub block(32字节key)
Level 0		
Level 1		
Level 2		
Level 3	6MB	24MB
Level 4	60MB	240MB
Level 5	600MB	2400MB
Level 6	6GB	24GB

其余cache用来做sst sub block的read cache，使用LRU淘汰

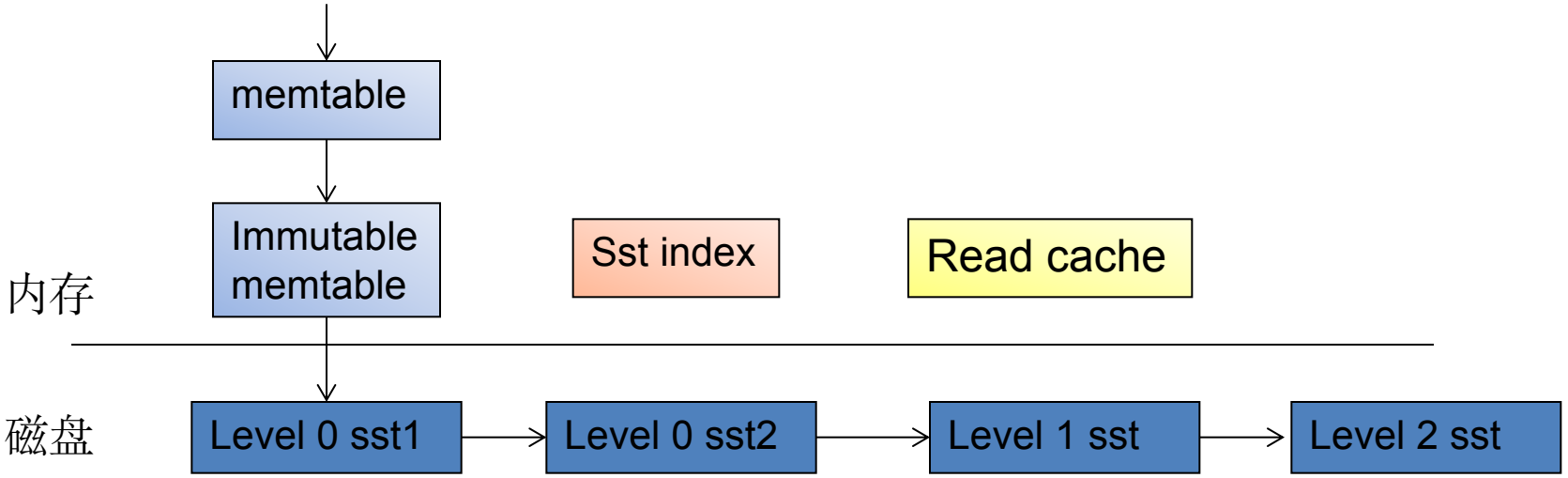
预期效果：

1.加入bloom filter以后，随机读时，读取磁盘中文件次数的数学期望是1.2次，基本上是读1-2次文件。

存储引擎——操作流程-读写删

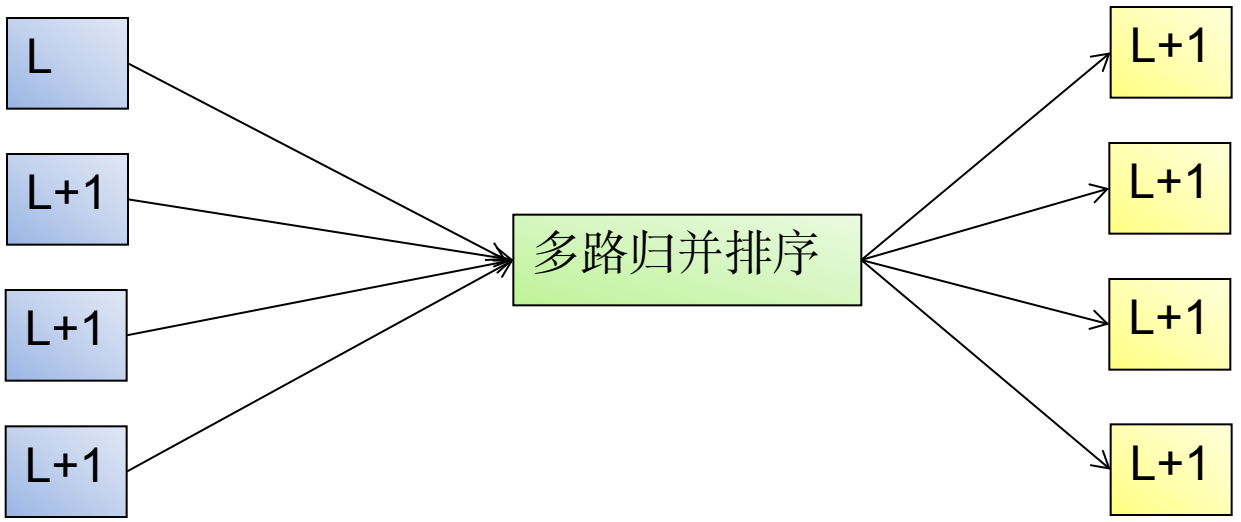
- 写入和删除操作都是直接向memtable插入一条记录即返回

- 读操作流程如下：



存储引擎——操作流程-合并操作（1）

- 内存到磁盘
  - Memtable 满时，将自己变成immutable memtable，然后将immutable memtable组装成一个sst block 放入level 0
- Level n 到 level n+1
  - Level n的sst block数目达到设定值后，启动本层向下层的合并
  - Level n(n>0)选择需要合并的sst block时，按range轮流选择1个sst block与下层range有重合的所有sst block进行合并
  - Level 0进行合并时，先选择1个level 0的文件，然后找出与改文件有重叠的所有level0 文件与level1的文件做合并



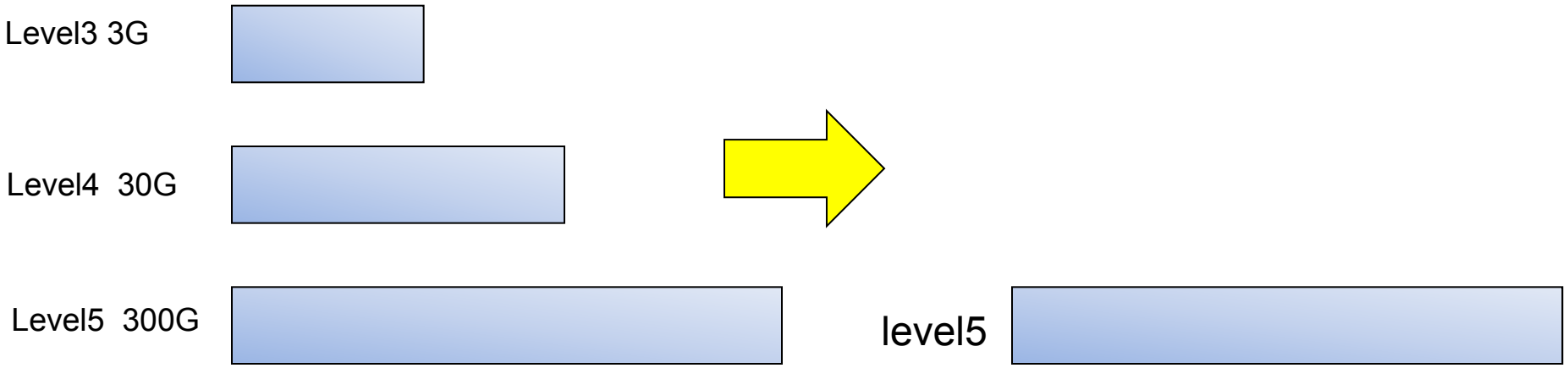
存储引擎——操作流程-合并操作（2）

- 自适应合并
  - 当用户请求压力不大时（磁盘util小于<10%），存储引擎可以将磁盘中的多层level合并为1层level。

  - 优势
  - 在用户的写高峰时期，提升用户的写带宽。

对于ssd 磁盘来说

- 在level3的3G数据写满前，用户可以获得约40MB/s的写带宽
- 在level4的30G数据写满前，用户可以获得约6MB/s的写带宽
- Level4的数据被写满后，用户只能获得3MB/s的写带宽



存储引擎——性能分析和对比

		Leveldb	B+ tree 16K page	Leveldb for SATA	tssdv3
单个block大小		1MB	16K	4MB	\
层数		3	8	4	\
key: 64 Byte value: 200 Byte	索引大小	400MB	1.2	600MB	1560MB
	随机读平均读取次数	1.2	1	1.16	1
	读放大	74 (=62*1.2)	62	io放大次数为 2*1.16	20
	写放大	12	62以上	18	40
key: 1024 Byte value: 4096 Byte	索引大小	400MB	20G （前面7层约2G，设cache大小为1G，可以缓存前面的6.5层）	600MB	1560MB
	随机读平均读取次数	1.2	2.5 (1.5次索引+1次数据)	1.16	1
	读放大	5 (=4*1.2)	4	io放大次数为 2*1.16	1
	写放大	12	5以上	18	2

资源调度

- 资源监控
- 分片均衡
- 资源均衡策略
  - 容量满调度
  - 访问量满调度
  - 容量均衡调度
  - 访问量均衡调度
- 分片迁移&合并&分裂机制

资源调度

容量满调度

目的：保证磁盘不会被写满

调度策略:

1. 找出分片容量之和超过磁盘容量阈值（85%）的磁盘列表
2. 从磁盘容量

数据均衡&迁移

- 均衡（细化）
  - 监控
  - 分裂
  - 调度

- 迁移（细化）
  - 粒度以分片（shard）为粒度进行
  - 流程与tssd类似

故障探测&恢复

- 探测
- 主从切换
- 恢复（调研，重点）
  - 对未同步记录的存储(流水日志）
  - 没有写Hint
  - 分range，设seq，比较